MinPause: Recovery-Preserving Emergency Control for DeFi Lending

Niko Rokni Lamouki[1*]

[1*]School of Architecture, Computing and Engineering, University of East London, Docklands Campus, London, E16 2RD, United Kingdom.

Corresponding author(s). E-mail(s): nrokni@uel.ac.uk;

Abstract

Emergency pauses can stop non-atomic decentralized-finance attacks, but a global freeze also blocks legitimate activity and may prevent borrowers from repaying debt or adding collateral. We formulate emergency response as a constrained, weighted hitting-set problem: select the least-cost set of protocol operations whose suspension intersects every identified attack path while designated recovery operations remain available. We implement *MinPause*, an exact and auditable policy synthesizer, and evaluate it over 360 deterministic Latin-hypercube scenarios and 2,160 policy–scenario evaluations. MinPause blocks 100.0% of supplied attack paths while retaining 85.6% of legitimate activity; omitting one input path reduces coverage to 94.3%, quantifying model-risk sensitivity. We prove path coverage under an explicit labelled-transition-system assumption, characterize feasibility, derive necessary and sufficient borrower-resource conditions for recovery, and prove NP-completeness of the decision problem. A controlled Solidity harness further verifies locally that pausing only an Euler-inspired `donateToReserves` selector blocks the hazardous transition while `repay` and `addCollateral` remain callable; the repository includes a Sepolia deployment workflow whose public-chain evidence is reported only after signed execution.

Keywords: Decentralized finance, emergency pause, circuit breaker, hitting set, borrower recovery, smart-contract security, Sepolia

1 Introduction

Decentralized lending protocols operate continuously, settle transactions quickly, and expose composable entry points to untrusted callers. When an incident is detected, a guardian can often pause a protocol faster than a governance upgrade can be proposed, audited, and executed. This response can be effective when an attack is non-atomic: Zhou et al. found that 56% of the attacks in their incident corpus left a potential rescue interval [1]. Yet pausing is not free. A global freeze exchanges integrity for availability and can prevent users from reducing their own exposure. In a lending market, blocking `repay` or collateral top-up during a price shock may turn a containable incident into avoidable liquidation or bad debt.

Production systems therefore expose controls at different granularities. OpenZeppelin's `Pausable` pattern guards selected state-changing operations [2, 3]; Compound-style guardians can suspend individual actions, and recent incident-response guidance distinguishes broad, function-level, and asset-level stops [4]. However, the policy determining *which* functions to stop is commonly written by hand. A manual policy may be too broad, leaving unnecessary denial of service, or too narrow, leaving an alternative attack path executable.

This paper asks: *Given a set of plausible attack paths, which minimum-cost operations should be paused so that every path is interrupted while borrower recovery remains possible?* We answer with MinPause, a transparent policy synthesizer built around an action–effect graph and a binary

optimization problem. Recovery-critical actions are hard constraints rather than soft preferences. If an identified path can only be interrupted by disabling a recovery-critical action, the model reports infeasibility instead of silently weakening the requirement.

The contributions are:

- a protocol-independent action–effect representation of lending operations, state effects, and alternative attack paths;
- the Minimum Recovery-Preserving Pause problem, combining path coverage, legitimate-use cost, and group-level disruption;
- an exact, deterministic solver and a public experimental artifact with 360 reproducible scenarios;
- a counterfactual comparison of four conventional responses with MinPause, including a missing-path sensitivity experiment and an Euler-inspired Solidity validation harness; and
- formal guarantees for represented-path coverage, feasibility, borrower recovery resources, and computational complexity.

2 Related Work

2.1 DeFi incidents and response

The DeFi security literature has catalogued smart-contract, economic, oracle, governance, and composability failures. The SoK by Zhou et al. combines 181 incidents with academic and audit evidence, studies rescue intervals, and measures the use and timing of emergency pauses [1]. Daian et al. show how transaction ordering and miner-extractable value create adversarial execution environments [5]; Qin et al. analyze how flash-loan atomicity amplifies attack strategies [6]. Separate empirical work documents liquidation design and borrower costs across major lending protocols [7]. These studies motivate rapid containment, but do not synthesize the scope of the containment action.

Emergency intervention also has economic effects. During the Ethereum Merge, Aave paused ETH borrowing while Compound altered interest parameters and borrowing caps. Heimbach et al. show that these interventions affected liquidity and redistributed hard-fork-arbitrage value [8]. More generally, rigid lending terms can amplify fragility and multiple-equilibrium behaviour [9]. MinPause focuses on the narrower operational question that precedes such welfare analysis: which actions must be disabled after an alert?

2.2 Circuit breakers and formal models

Industry guidance recommends monitoring, pause roles, function limits, rate limits, and rehearsed recovery procedures [2, 10]. OpenZeppelin notes that a pause can be broad or granular and that a broad pause creates a denial-of-service state for legitimate users [2]. Formal work has represented the circuit-breaker design pattern with Dynamic Condition Response graphs, providing an unambiguous behavioural specification [11]. That work models how a pause behaves; MinPause instead chooses a pause set from competing security and availability requirements.

The closest combinatorial abstraction is weighted hitting set: every attack path is a set, and a feasible pause policy must intersect every such set. Hitting-set formulations are well understood [12], but their use with hard recovery constraints and borrower-level counterfactual evaluation is, to our knowledge, absent from the DeFi pause literature. This novelty claim is intentionally narrow: MinPause does not claim to invent pausing, attack graphs, or hitting set individually.

3 System and Threat Model

3.1 Protocol state

Let F be the finite set of independently pausable operations. Our reference market uses

$$F = \{\mathsf{supply}, \mathsf{withdraw}, \mathsf{borrow}, \mathsf{repay}, \mathsf{addCollateral}, \\ \mathsf{liquidate}, \mathsf{flashLoan}, \mathsf{swap}, \mathsf{oracleUpdate}\}.$$

The abstraction is not tied to a particular implementation: an operation can represent one Solidity entry point, several entry points sharing a pause flag, or an external adapter that governance can isolate.

Borrower i has post-shock state

$$s_i(t) = \big(C_i(t), D_i(t), \ell_i, A_i^D, A_i^C, g_i\big),$$

where C_i and D_i are collateral and debt values, ℓ_i is the liquidation threshold, A_i^D is immediately repayable debt, A_i^C is available additional collateral, and g_i is a user-size group. The health factor is

$$H_i(t) = \frac{\ell_i C_i(t)}{D_i(t)}. \tag{1}$$

A position is at risk when $H_i < 1$.

3.2 Attack paths and responder

An attack path $P_k \subseteq F$ is the operation support of one executable route to a prohibited economic effect. Multiple paths may reach the same effect. The detector supplies a family $\mathcal{P} = \{P_1, \ldots, P_m\}$; path extraction is deliberately outside the decision engine. In deployment, MinPause consumes alerts and traces from monitoring or transaction-analysis systems such as Forta or Phalcon and returns a candidate cut set [13, 14]. The separation is important: the optimizer guarantees coverage only relative to the paths supplied to it, not discovery of an unknown exploit.

The responder controls operation-level pause flags but cannot reverse settled transactions, change historical oracle observations, or stop calls that finish atomically before detection. We assume the guardian is uncompromised, pause transactions are eventually included, and every represented path contains at least one independently pausable non-recovery operation. Because these assumptions can fail, the initial operating mode is shadow recommendation with human guardian confirmation. Section 7.4 quantifies the effect of deliberately omitting one path.

4 MinPause Framework

4.1 Decision variables and constraints

For each $f \in F$, let

$$x_f = \begin{cases} 1, & f \text{ is paused,} \\ 0, & f \text{ remains active.} \end{cases}$$

Every identified path must contain a paused operation:

$$\sum_{f \in P_k} x_f \geq 1, \qquad k = 1, \ldots, m. \tag{2}$$

Let $R \subseteq F$ be the recovery-critical set. In the implementation, $R = \{\mathsf{repay}, \mathsf{addCollateral}\}$, and

$$x_r = 0, \qquad r \in R. \tag{3}$$

Each operation has normalized legitimate-use cost $c_f \geq 0$. Group-specific costs c_{fg} represent the share borne by group g. For a policy x, group burden is

$$B_g(x) = \frac{\sum_f c_{fg} x_f}{\sum_f c_{fg}},$$

and disparity is $\Delta(x) = \max_g B_g(x) - \min_g B_g(x)$. MinPause solves

$$\min_{x \in \{0,1\}^{|F|}} \sum_{f \in F} c_f x_f + \lambda \Delta(x) \quad \text{s.t. } (2)\text{–}(3). \tag{4}$$

We use $\lambda = 0.15$ in the main experiment. The fairness term is a design choice, not a claim that the three size groups capture all meaningful welfare differences.

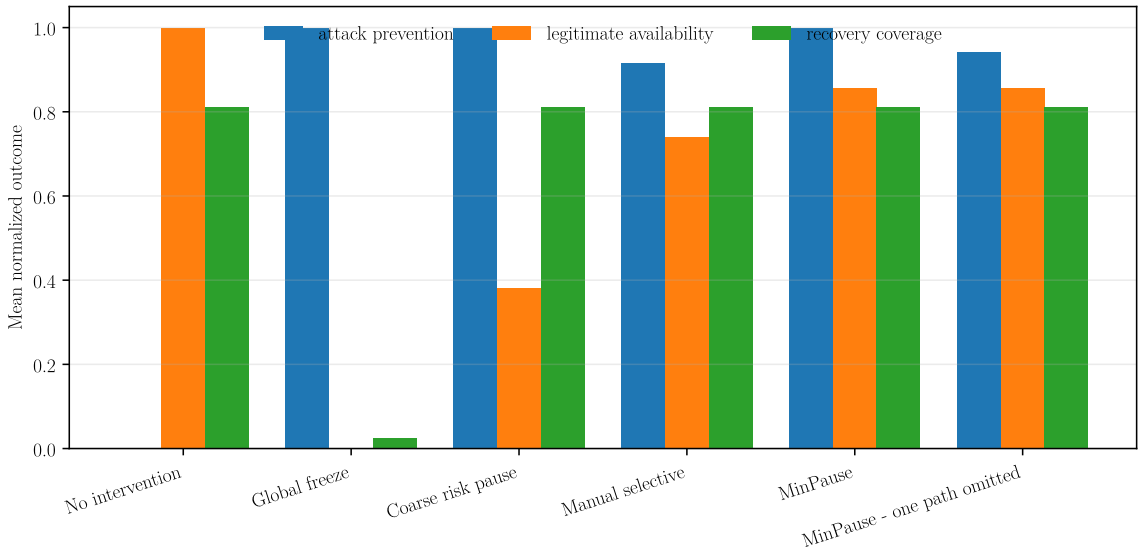


Fig. 1 Mean security, availability, and borrower-recovery outcomes across 360 scenarios.

4.2 Exact synthesis algorithm

Because a market normally exposes only a small number of independent pause flags, the reference implementation enumerates all subsets of $F \setminus R$. For each subset it checks path coverage, computes the objective, and retains the minimum with deterministic tie-breaking by cardinality and operation name. With nine operations and two protected recovery operations, at most $2^7 = 128$ subsets are evaluated per incident. This exact implementation is easier to audit than a heuristic and removes solver-specific variability.

The output contains the paused set, feasibility flag, legitimate-use cost, disparity, and objective value. An infeasible result is operationally meaningful: the identified threat cannot be contained under the declared recovery guarantee, so the responder must relax the guarantee, upgrade, migrate, or use a broader emergency procedure.

4.3 Scalability extension

Enumeration is intentional for one market: nine operations and two protected recovery operations yield only $2^7 = 128$ candidates, making the result deterministic and directly auditable. For composable systems with $|F| > 30$, Eq. (4) is a 0–1 mixed-integer linear program. Introduce $u \geq B_g(x)$ and $v \leq B_g(x)$ for every group g, then minimize $\sum_f c_f x_f + \lambda(u - v)$ subject to Eqs. (2)–(3). Standard branch-and-bound solvers such as CBC or Gurobi can then replace enumeration. Runtime depends on path structure and must be benchmarked for the target ecosystem; we do not claim a universal sub-second bound.

5 Formal Analysis

Definition 1 (Labelled protocol semantics)**.** *A protocol is a labelled transition system $\mathcal{T} = (S, F, \to)$, where S is the state space and $s \xrightarrow{f} s'$ denotes a transition caused by operation $f \in F$. A trace is an ordered sequence $\sigma = \langle f_1, \ldots, f_q \rangle$ for which states $s_0, \ldots, s_q$ exist with $s_{j-1} \xrightarrow{f_j} s_j$. Its support is* $\mathrm{supp}(\sigma) = \{f_j : 1 \leq j \leq q\}$*. For a pause set $X \subseteq F$, $\mathcal{T}_X$ removes all transitions labelled by elements of X.*

The catalogue is *complete for represented harm* if every harmful trace in its stated incident class is represented by a path $P_k = \mathrm{supp}(\sigma_k)$. We additionally assume hard dependency: every step of σ_k is necessary for that represented execution. In one atomic transaction, failure of a required call reverts the transaction; across transactions, failure of a required predecessor prevents a later causal precondition. This assumption excludes optional calls from P_k and makes explicit where the unordered set abstraction is sound.

Theorem 1 (M1: represented-path coverage)**.** *If $X \cap P_k \neq \varnothing$ for every $P_k \in \mathcal{P}$, no catalogued trace σ_k is executable in $\mathcal{T}_X$. Under catalogue completeness, no harmful trace in the represented incident class is executable.*

Proof. Fix k and choose $f \in X \cap P_k$. Because $P_k = \mathrm{supp}(\sigma_k)$, the ordered trace contains a transition labelled f. That transition is absent from $\mathcal{T}_X$, so the complete hard-dependent trace cannot execute. The argument holds for every k. Catalogue completeness extends the conclusion from listed traces to the stated incident class. It does not cover a trace omitted by the detector. □

Theorem 2 (M2: feasibility)**.** *Assume every $f \in F \setminus R$ is independently pausable and there are no constraints beyond Eqs. (2)–(3). MinPause is feasible if and only if*

$$\forall P_k \in \mathcal{P}, \qquad P_k \setminus R \neq \varnothing. \tag{5}$$

Proof. ($\Rightarrow$) Suppose a feasible vector exists but some $P_k \subseteq R$. Equation (3) gives $x_f = 0$ for every $f \in P_k$, hence $\sum_{f \in P_k} x_f = 0$, contradicting Eq. (2). ($\Leftarrow$) If Eq. (5) holds, set $x_f = 1$ for all $f \in F \setminus R$ and $x_r = 0$ for $r \in R$. Every path contains a member of $F \setminus R$, so every coverage constraint is satisfied. Empty paths are therefore correctly classified as infeasible. □

Theorem 3 (M3: resource-bounded recovery)**.** *Fix $C_i \geq 0$, $D_i > 0$, and $0 < \ell_i \leq 1$. Let $0 \leq a_D \leq A_i^D < D_i$ and $0 \leq a_C \leq A_i^C$. A pair of admissible actions can restore $H_i' \geq 1$ if and only if*

$$\ell_i(C_i + A_i^C) \geq D_i - A_i^D. \tag{6}$$

When Eq. (6) holds, recovery is reachable in one aggregated repayment/top-up action under fixed prices and atomic execution of the selected calls.

Proof. For $a_D < D_i$,

$$H_i' = \frac{\ell_i(C_i + a_C)}{D_i - a_D} \geq 1 \quad \Longleftrightarrow \quad \ell_i(C_i + a_C) \geq D_i - a_D.$$

The left side increases with a_C and the right side decreases with a_D. More explicitly,

$$\frac{\partial H_i'}{\partial a_C} = \frac{\ell_i}{D_i - a_D} > 0, \qquad \frac{\partial H_i'}{\partial a_D} = \frac{\ell_i(C_i + a_C)}{(D_i - a_D)^2} \geq 0.$$

Therefore the maximum attainable health factor uses $(a_D, a_C) = (A_i^D, A_i^C)$, proving necessity and sufficiency of Eq. (6). Executing that feasible pair reaches the target in one aggregated step. If full repayment $A_i^D \geq D_i$ is permitted, choosing $a_D = D_i$ eliminates debt and is a separate trivially safe boundary case rather than a division by zero. □

MinPause guarantees only that repay and addCollateral remain callable. Theorem M3 shows why this is not universal economic recovery: a user who fails Eq. (6) lacks sufficient resources despite callable functions. With per-transaction caps, finite reachability additionally requires positive caps and stable valuations; batching the feasible totals then takes finitely many calls.

Theorem 4 (M4: computational complexity)**.** *The decision version of MinPause with $R = \varnothing$ and $\lambda = 0$ is NP-complete. Consequently, the optimization problem is NP-hard.*

Proof. Define the decision problem: given F, path family $\mathcal{P}$, non-negative integer costs c_f, and budget K, determine whether some $X \subseteq F$ satisfies $X \cap P_k \neq \varnothing$ for all k and $\sum_{f \in X} c_f \leq K$. A proposed binary vector is verified by summing its cost and checking all incidences in time polynomial in $|F| + \sum_k |P_k|$, so the problem is in NP.

For NP-hardness, take an arbitrary Minimum Weight Hitting Set decision instance with universe U, family $\mathcal{S}$, weights w_u, and budget K. Construct a MinPause instance with $F = U$, $\mathcal{P} = \mathcal{S}$, $c_f = w_f$, $R = \varnothing$, and $\lambda = 0$. The construction is linear in the input representation. A subset X hits every member of $\mathcal{S}$ with weight at most K if and only if the identical subset is a feasible pause set with objective at most K. Thus the decision problems are equivalent, proving NP-hardness and, with membership in NP, NP-completeness. An exact optimizer for the general problem would solve the decision problem, so optimization is NP-hard. □

Table 1 Mean results across 360 scenarios. Higher is better for attack prevention, availability, and recovery; lower is better for policy size and disparity.

Policy	Attack prevention	Availability	Recovery coverage	Policy size	Disparity
No intervention	0.0%	100.0%	81.2%	0.00	0.000
Global freeze	100.0%	0.0%	2.5%	9.00	0.000
Coarse risk pause	100.0%	38.0%	81.2%	5.00	0.334
Manual selective	91.7%	74.0%	81.2%	3.00	0.058
MinPause	**100.0%**	**85.6%**	**81.2%**	**1.34**	**0.042**
MinPause–one path omitted	94.3%	85.7%	81.2%	1.29	0.033

6 Experimental Methodology

6.1 Design and reproducibility

We generate 360 configurations using a seeded, jittered Latin hypercube [15]. The sampled dimensions are collateral price shock $[0.02, 0.45]$, utilization $[0.35, 0.98]$, detection delay $[1, 90]$ minutes, potential attack loss $[0.25, 20]$ million USD on a logarithmic scale, liquidity $[1, 100]$ million USD on a logarithmic scale, borrower count $[120, 600]$, recovery capacity $[0.04, 0.50]$, activity skew, and incident family. Four incident families—oracle, liquidity, accounting, and composite—contain three to five alternative attack paths.

Borrower debt is lognormally distributed, initial health factors lie in $[1.03, 1.85]$, and liquidation thresholds lie in $[0.72, 0.86]$. A price shock is applied before policy evaluation. Available repayment and collateral top-up are sampled as bounded shares of each position. These synthetic distributions create controlled counterfactuals; they are not estimates of a named production market.

Every scenario is evaluated under six policies:

1. **No intervention**: all operations remain active;
2. **Global freeze**: all nine operations are disabled;
3. **Coarse risk pause**: withdraw, borrow, liquidate, flash loan, and swap are disabled;
4. **Manual selective**: borrow, flash loan, and oracle update are disabled; and
5. **MinPause**: the solution of (4); and
6. **MinPause–one path omitted**: MinPause synthesized after removing the final deterministic catalogue path, then evaluated against the complete path family.

The complete design yields 2,160 policy–scenario evaluations. The repository stores scenario-level CSV output, aggregate summaries, figures, fixed seeds, and automated tests.

6.2 Metrics

Attack prevention is the fraction of identified paths intersected by the pause set. Detection delay determines the preventable share of potential loss through $e^{-\tau/55}$; this reduced-form decay is used consistently across policies and is varied through τ. Legitimate availability is one minus the normalized pause cost. Recovery coverage is the share of post-shock at-risk borrowers able to reach $H_i \geq 1$ using whichever of repayment and collateral top-up remain callable, subject to their sampled resource limits. We also report policy cardinality, bad-debt proxy, liquidation count, deferred risky positions, and group-burden disparity.

7 Results

7.1 Security and availability

Table 1 and Fig. 1 show the central trade-off. Global freeze and coarse risk pause cover every modelled path, but retain 0.0% and 38.0% of legitimate activity, respectively. The fixed manual policy retains 74.0% availability but covers only 91.7% of paths. MinPause covers 100.0% while pausing only 14.4% of normalized legitimate activity. Its mean policy size is 1.34 operations.

The result does not imply that smaller policies are universally safer. It follows from the supplied path family: if a relevant path is absent or incorrectly abstracted, any optimizer can return a policy that is precisely optimal for the wrong model. MinPause's benefit is auditability—the coverage obligation and selected cut are explicit.

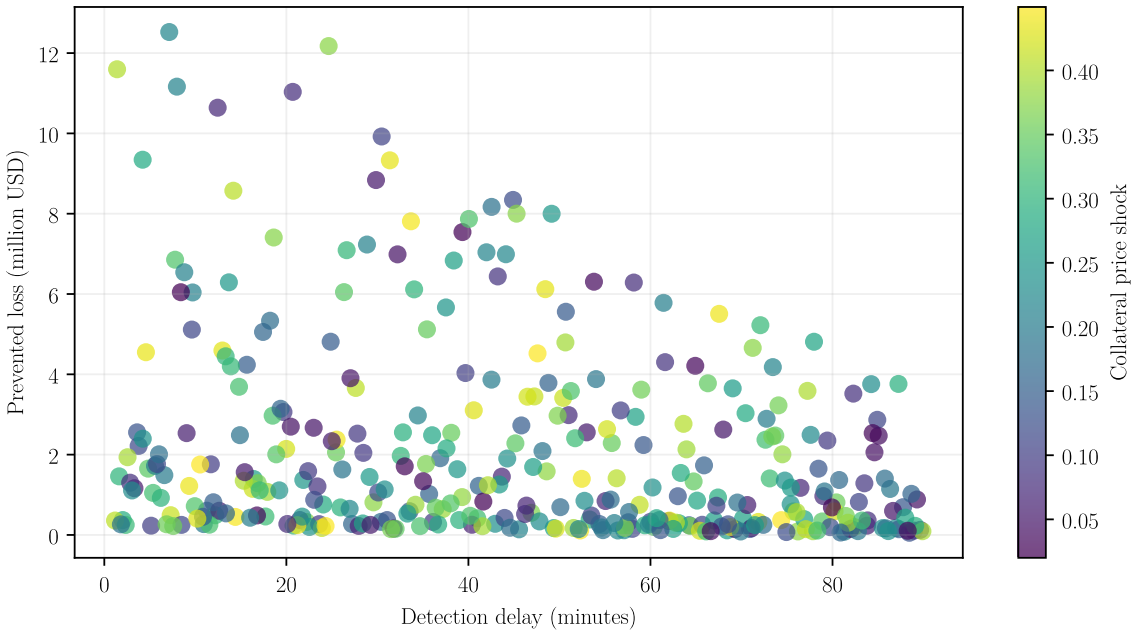


Fig. 2 MinPause prevented loss across detection delays and collateral price shocks. Policy coverage remains complete; the economically preventable amount declines with delay.

7.2 Recovery and distributional effects

Recovery coverage under MinPause, the coarse pause, and the manual policy is 81.2%. The remaining borrowers lack sufficient sampled resources to restore $H_i \geq 1$ even though recovery functions are callable. A global freeze reduces mean recovery coverage to 2.5% because both recovery operations are unavailable; the nonzero mean arises from scenarios with no post-shock at-risk accounts. Thus, preserving a function is necessary but not sufficient for economic recovery.

The coarse risk pause produces the largest mean burden disparity (0.334), reflecting its broad suppression of operations used differently across retail, medium, and whale profiles. MinPause reduces this to 0.042. This result should be interpreted as a model diagnostic rather than a social-welfare conclusion because address size is an incomplete proxy for user type.

7.3 Robustness over the sampled space

MinPause maintains full path coverage in every feasible generated scenario; variation in prevented loss is driven by the potential loss and detection delay rather than by missed paths. Figure 2 visualizes this distinction. Long delays leave less loss preventable even when the selected policy is logically sufficient. Emergency policy synthesis therefore complements, rather than replaces, low-latency monitoring and transaction inclusion.

7.4 Missing-path sensitivity

Deleting one path before synthesis while retaining the full family for evaluation reduces mean attack-path coverage from 100.0% to 94.3%. Mean realized attack loss rises from USD 2.44 million to USD 2.57 million, even though the incomplete model reports a slightly smaller policy (1.29 rather than 1.34 operations). This is the expected failure mode of a decision engine: optimization cannot compensate for missing threat information. The result motivates shadow-mode deployment, conservative path unions from multiple detectors, and guardian review of the selected cut before automation.

7.5 Controlled Solidity validation

We implement an Euler-inspired test fixture in Solidity rather than claiming a replay of Euler's production state. Euler's own incident account and a transaction-level debugger identify a missing health/liquidity check in `donateToReserves` as the enabling condition [16, 17]. The fixture deliberately reproduces that control-surface omission. In the unprotected run, donating 800 of 1,000 collateral units after borrowing 700 moves the health factor from 1.143 to 0.229. In the protected

run, the guardian pauses only the `donateToReserves(uint256)` selector; the hazardous call reverts, while a 100-unit repayment and 100-unit collateral top-up remain callable and raise the health factor to 1.467. Both assertions pass in the deterministic Hardhat suite. A GitHub Actions workflow deploys the same controlled fixture to Sepolia and records addresses, transaction hashes, blocks, selectors, and assertions. Because no signing key is embedded in the artifact, public-chain execution is reported only after the workflow produces a verifiable evidence file. This test establishes enforcement and deployability, not historical Euler loss prevention.

8 Discussion

8.1 Design implications

The experiments support three implementation recommendations. First, protocols should expose distinct pause domains rather than one undifferentiated global flag. Second, repayment and collateral top-up should be explicitly tested in the paused state; calling them "allowed" in documentation is insufficient if a shared internal transfer guard makes them revert. Third, the action–effect model and candidate policies should be prepared and rehearsed before an incident. Constructing the model while funds are leaving defeats the latency advantage of a circuit breaker.

MinPause should initially operate in shadow mode. Given an alert, it can emit a recommended policy and explanation while the guardian retains control. Historical replay and fork tests should compare the recommendation with known incidents. Automated execution is only defensible after path-extraction error, false alerts, role compromise, and integration failures are measured.

8.2 Limitations

The economic evaluation is simulation-based and does not claim production deployment or mainnet-fork validation. The Solidity experiment uses a controlled Euler-inspired fixture on a mock asset; a Sepolia run proves public-testnet execution only after signed transaction evidence is archived. Attack paths are supplied rather than automatically discovered. The reduced-form loss curve, synthetic borrower distribution, and normalized pause costs are controlled assumptions. Composability can move harm outside the modelled protocol, and a pause transaction can be censored, front-run, or arrive after an atomic exploit. The solver protects only declared recovery actions and does not prove that external tokens, bridges, or oracles required by those actions remain live. Theorems M1–M3 are conditional on catalogue completeness, independent pause flags, fixed valuations, and user resource bounds; they do not remove these modelling assumptions.

9 Conclusion

MinPause reframes emergency control as a constrained selection problem rather than an all-or-nothing switch. Across 360 reproducible scenarios, it covers all supplied attack paths while preserving 85.6% of legitimate activity and keeping borrower recovery operations callable. The formal analysis states exactly what follows from path coverage and what remains dependent on detector completeness and borrower liquidity. The controlled Solidity test demonstrates function-level enforcement without conflating it with historical validation. The next empirical step is a pinned Euler mainnet-fork replay and comparison of paths produced independently by multiple monitoring systems.

Reproducibility statement

The optimizer is in `src/minpause/optimizer.py`; scenario generation is in `src/minpause/scenarios.py`; policy evaluation is in `src/minpause/simulation.py`; the full run is reproduced by `scripts/run_experiments.py`; and tests are in `tests/`. Generated CSV files are stored under `results/` and figures under `figures/`.

Declarations

Funding. No external funding was received for this study. **Competing interests.** The author declares no competing interests. **Data and code availability.** Source code, generated scenarios, aggregate results, figures, Solidity fixtures, and deployment workflows are provided in the accompanying public repository.

References

[1] Zhou L, Xiong X, Ernstberger J, Chaliasos S, Wang Z, Wang Y, et al. SoK: Decentralized Finance (DeFi) Attacks. In: 2023 IEEE Symposium on Security and Privacy; 2023. .

[2] OpenZeppelin.: Smart Contracts and Incident Response: Insight on Current Mechanisms. Accessed November 2026. https://www.openzeppelin.com/news/smart-contracts-incident-response-insight-on-current-mechanisms.

[3] OpenZeppelin.: Contracts Utilities: Pausable. Accessed November 2026. https://docs.openzeppelin.com/contracts/5.x/api/utils.

[4] Security OSC.: SCWE-156: Missing Emergency Circuit Breaker for Critical Operations. Accessed November 2026. https://scs.owasp.org/SCWE/SCSVS-GOV/SCWE-156/.

[5] Daian P, Goldfeder S, Kell T, Li Y, Zhao X, Bentov I, et al. Flash Boys 2.0: Frontrunning in Decentralized Exchanges, Miner Extractable Value, and Consensus Instability. In: 2020 IEEE Symposium on Security and Privacy; 2020. p. 910–927.

[6] Qin K, Zhou L, Livshits B, Gervais A. Attacking the DeFi Ecosystem with Flash Loans for Fun and Profit. In: Financial Cryptography and Data Security; 2021. p. 3–32.

[7] Qin K, Zhou L, Gamito P, Jovanovic P, Gervais A. An Empirical Study of DeFi Liquidations: Incentives, Risks, and Instabilities. In: Internet Measurement Conference; 2021. p. 336–350.

[8] Heimbach L, Schertenleib E, Wattenhofer R. DeFi Lending During The Merge. In: 5th Conference on Advances in Financial Technologies. vol. 282 of LIPIcs; 2023. p. 9:1–9:25.

[9] Chiu J, Ozdenoren E, Yuan K, Zhang S. On the Fragility of DeFi Lending. Centre for Economic Policy Research; 2025. DP20434.

[10] Security OSC.: DeFi Recovery Patterns: Pause and Upgrade. Accessed November 2026. https://scs.owasp.org/handbooks/04-evm-forensics-defi-recovery/part4-defi-recovery-patterns/.

[11] Eshghie M, Ahrendt W, Artho C, Hildebrandt TT, Schneider G. Formalizing Smart Contract Design Patterns with DCR Graphs. Software and Systems Modeling. 2026;25:1083–1121.

[12] Vazirani VV. Approximation Algorithms. Springer; 2001.

[13] Foundation F.: DeFi Threat Detection Kit. Accessed November 2026. https://docs.forta.network/en/latest/defi-starter-kit/.

[14] BlockSec.: Phalcon Explorer. Accessed November 2026. https://docs.blocksec.com/phalcon/phalcon-explorer.

[15] McKay MD, Beckman RJ, Conover WJ. A Comparison of Three Methods for Selecting Values of Input Variables in the Analysis of Output from a Computer Code. Technometrics. 1979;21(2):239–245. https://doi.org/10.1080/00401706.1979.10489755.

[16] Labs E.: War & Peace: Behind the Scenes of Euler's $240M Exploit Recovery. Accessed November 2026. https://euler.finance/blog/war-peace-behind-the-scenes-of-eulers-240m-exploit-recovery.

[17] BlockSec.: Phalcon Explorer Debugger: Euler Exploitation Transaction. Accessed November 2026. https://docs.blocksec.com/phalcon/phalcon-explorer/debugger.